Web Mining – Short Exam Notes (Units I & II)

UNIT I

1. Web Mining basics

- **Web mining** = data mining applied to web **content, links, and usage logs**.
- **Process:** Raw data → Patterns → Knowledge.
- **Challenges:** Too large · Complex/unstructured pages · Dynamic · Relevancy (user cares about tiny part).
- **Applications:** personalised marketing, e-commerce recommendations, SEO, fraud detection, sentiment analysis.

Data Mining vs Web Mining

	Data Mining	Web Mining
Data	Structured (tables)	Structured + semi + unstructured
Problems	Clustering, classification, regression	Content / structure / usage mining
Tools	ML algorithms	Scrapy, PageRank, Apache logs

> **Web mining is the child; data mining is the parent.**

2. Association Rules (Apriori, Eclat, FP-Growth)

- **Rule:** `Bread → Butter` (antecedent → consequent). Finds items that occur **together**.

3 metrics - **Support** = (txns with A and B) ÷ (total txns) → *how common* - **Confidence** = Support(A∪B) ÷ Support(A) → *how reliable* - **Lift** = Confidence ÷ Support(B) → *how real* (>1 positive, =1 none, <1 negative)

Quick numerical (5 txns): Bread+Butter in 3 → Support 3/5 = 0.6. Bread in 4 → Confidence 3/4 = 0.75. Support(Butter)=0.6 → Lift 0.75/0.6 = **1.25**.

Apriori (Agrawal & Srikant 1994) – level by level (1-items → pairs → triples). - **Apriori property:** every subset of a frequent itemset is frequent → prune anything with an infrequent subset. - Steps: count singles → join to make candidates → prune → scan & count → repeat. - **Limitations:** huge candidate sets, many DB scans (n+1).

	Apriori	Eclat	FP-Growth
Search	BFS	DFS	Tree, divide & conquer
Data	Horizontal	Vertical (tid-lists)	FP-tree
Candidates	Yes	Yes (intersection)	**No**
DB scans	Many	Few	**Two**
Speed	Slowest	Medium	Fastest

- **Eclat** = Equivalence CLAss Transformation; support = intersect tid-lists.

- **FP-Growth** = Frequent Pattern tree; pass 1 counts, pass 2 builds tree (items sorted by frequency); mine from least frequent item upward.

---

3. Sequential Patterns & GSP

- **Order matters:** `<a b>` ≠ `<b a>`. Inside one element order does not matter: `(bc)` = `(cb)`.
- `<b c>` = different transactions; `<(bc)>` = same transaction.
- **Terms:** Sequence, Element (one transaction), Item, Subsequence (same order, not necessarily consecutive), k-length sequence (k items total).

GSP (Generalized Sequential Pattern) = Apriori for sequences. 1. Count single items → L1 2. **Join:** drop first item of s1 and last of s2; if remainder identical, join. `<a b>` + `<b e>` → `<a b e>` 3. **Prune:** remove candidate if any (k-1)-subsequence is infrequent 4. Count support, repeat. - **Drawback:** many scans, many candidates. - Methods: Apriori-based (GSP, SPADE); Pattern-growth (FreeSpan, PrefixSpan).

> **Association = "together"; Sequential = "one after another".**

---

4. Machine Learning in Data Mining

Type	Labels?	Algorithms	Web example
Supervised (classification)	Yes	Decision Tree, Naive Bayes, SVM, KNN, Logistic Reg	Spam / not spam
Unsupervised (clustering)	No	K-Means, Hierarchical	Group similar users
Semi-supervised	Partly	Co-training, self-training	Few labelled pages

> **Labels given = classification; no labels = clustering.**

---

5. Three Types of Web Mining

	Content	Structure	Usage
Mines	What's ON the page	Links BETWEEN pages	What users DO
Data	Text, images, video	Hyperlinks, HTML tags	Server logs, cookies
Also called	Text mining	Link mining	Log mining
Techniques	NLP, classification	PageRank, HITS	Assoc. rules, clustering

Web Usage Mining - Data sources: web server logs (IP, browser), application server data, application-level data. Collected server/client/proxy side. - Techniques: association rules (pages visited together), classification (frequent vs non-frequent buyer), clustering (user clusters, page clusters). - Applications: personalisation (Server → Logs → Pre-processing → Model → Analysis), e-commerce, prefetching/caching. - **Disadvantage:** privacy.

---

6. Web Structure Mining, Hubs & Authorities

- Web as **directed graph**: node = page, edge = hyperlink. In-degree = links IN, out-degree = links OUT.
- Process: Collect → Pre-process → Knowledge discovery → Analysis.

- Two types: **hyperlink level** (between pages: PageRank, HITS) vs **document level** (inside page: DOM tag tree).

HITS - **Hub** = points OUT to many good pages (directory page). - **Authority** = pointed IN by many good hubs (authentic content). - Good hub → many good authorities; good authority ← many good hubs.

---

7. PageRank

- Importance of a page from **number and quality of incoming links**. Google, Larry Page.
- Link = vote; vote from important page counts more; a page splits its vote across its out-links.

PR(A) = (1 – β) + β × [PR(B)/Cout(B) + PR(C)/Cout(C) + …] - B, C… = pages linking TO A; Cout = out-links; β = damping ≈ 0.8/0.85.

Steps: start all = 1/N → for each page, sum (rank of in-linker ÷ its out-links) → ×β → +(1−β) → repeat.

Example (β=0.8, 6 pages, start 0.1667): PR(A) = 0.2 + 0.8 × (0.1667/4 + 0.1667/2) = **0.3**; PR(B) = 0.2 + 0.8 × (0.3/2) = **0.32**.

> **IN-links decide who votes for you; OUT-links decide how much each vote is worth. Put brackets around the whole sum before ×β.**

UNIT II

8. Web Content Mining & Text Pre-processing

- Flow: Web pages → Pre-processing → Vector creation → Structured info.
- **Approaches:** Agent-based (intelligent search agents, info filtering, personalised agents) vs Database (multilevel DB, web query systems).
- **Text mining process:** Pre-processing → Transformation → Feature selection → Data mining → Evaluation.

Pre-processing steps: Noise removal → Segmentation → Tokenization → Lowercase → Spell correction → Stop-word removal → Stemming → Lemmatization → POS tagging.

	Stemming	Lemmatization
Method	Cuts endings by rules (Porter)	Dictionary + context
Output	May not be a real word (caring→car)	Always real word (caring→care)
Speed	Fast	Slow

- Web-page extras: strip HTML (weight title/headings/anchor), drop nav/ads, detect duplicates.

---

9. BoW, Vector Space Model, TF-IDF

- **Bag of Words:** vector of word counts, order ignored. Drawbacks: growing vocab, sparse, loses order.
- **VSM:** each term = one axis; document = point. Similarity = **cosine** = (d1·d2) ÷ (|d1|×|d2|). 1 = same, 0 = nothing common.

TF-IDF - TF = count of term in doc ÷ total terms in doc - IDF = log10(total docs ÷ docs containing term) - TF-IDF = TF × IDF → high when frequent in this doc but rare elsewhere. - Remember: log 1 = 0, log 1.5 ≈

0.18, log 3 ≈ 0.48. - Example (3 docs, Review 2 has 8 words): "not" appears in 1 doc → TF 1/8, IDF 0.48, TF-IDF **0.06**; "this" in all 3 → IDF 0 → **0**.

	BoW	TF-IDF
Value	Raw count	Weighted score
Common words	High	Pushed to 0

---

10. Dimensionality Reduction

- **Curse of dimensionality:** more features → complex model, more data needed, overfitting.
- Benefits: less storage/time, easier visualisation, removes redundancy.

	Feature Selection	Feature Extraction
Idea	Keep subset of originals	Build new features
Techniques	Variance threshold, correlation threshold, genetic algorithm, stepwise regression	PCA, LDA, t-SNE, autoencoders

- **PCA** (unsupervised, linear): covariance matrix → eigenvectors/values → transform → keep top components. Normalise first.
- **LDA** supervised; **t-SNE** non-linear for 2D/3D visualisation; **Autoencoder** = encoder → bottleneck → decoder.

> **Selection = choose from old; Extraction = build new.**

---

11. Inverted Index

- Maps **word** → **list of documents** (forward index is doc → words). Like a book's back index; hashmap: key = term, value = posting list.
- Search "lazy dog" = intersect lists of *lazy* and *dog*.
- **Types:** Record-level (doc IDs only, smaller) vs Word-level (doc + position, supports phrase search).
- **Build:** tokenize → remove stop words → stem (Porter) → record doc IDs (+ freq/position).
- **Pros:** fast full-text search, easy, standard in search engines. **Cons:** storage overhead, costly updates, no ranking by itself.

---

12. Latent Semantic Indexing (LSI/LSA)

- Finds **hidden meaning** by word co-occurrence; matches by meaning not exact words. Handles **synonyms** (car/automobile) and **polysemy** (bank).
- Flow: Pre-process → Document-Term Matrix → **SVD** → topic space → cosine similarity.
- $\mathbf{A = U \times \Sigma \times V^T}$ → U = word-topic, Σ = topic strengths (singular values), V^T = doc-topic. Keep top k.
- Pros: meaning, synonyms, noise reduction. Cons: ignores order, SVD slow, static (recompute for new docs), not real-time.
- Uses: search, clustering, plagiarism detection, QA.

> **LSI = term-document matrix + SVD.**

---

13. Web Spamming

- Tricking a search engine into ranking a page higher than deserved.

Type	Attacks	Techniques
Content/term spam	Relevance	Keyword stuffing (title, meta, body, anchor), copying text
Link spam	PageRank	Link farms, paid links, comment spam, link exchange
Hiding	Both	Hidden text, **cloaking** (different page to crawler vs user), redirection

- **Detection:** content analysis, link analysis, **TrustRank** (spread trust from verified good pages), ML classifier, fetch as crawler and as browser to catch cloaking.

---

14. Social Network Analysis (SNA)

- Graph theory on people: nodes = people, edges = relationships. **Sociogram** = graph; **Sociomatrix** = matrix.

	Ego network	Complete network
Focus	One person + direct contacts	Everyone in a bounded group
Data	Surveys, random sample	Every pair in the group
Measures	Size, diversity, density	Centrality, subgroups, equivalence

Centrality: Degree (most connections) · Closeness (reaches everyone fastest) · Betweenness (bridge on shortest paths). - Web = social network: pages = actors, links = relations → PageRank/HITS are SNA.

---

15. Web Crawlers

- Bot that downloads pages and follows links to index them. Web = directed graph. **Crawling = BFS.**
- **Algorithm:** seed URL in queue + visited set → pop front → download HTML → extract URLs (regex) → add unseen to visited + queue → repeat till empty.
- Data structures: **Queue (FIFO / LinkedList)** for to-crawl, **Visited set (HashSet)** to avoid repeats.
- **Politeness policy:** delay between requests, obey robots.txt.
- **Freshness** (copy still matches live page; check via HTTP HEAD) vs **Age** (days since last crawl). Updates follow Poisson; age is the better recrawl guide.
- Types: universal, focused (one topic), incremental (keeps refreshing).

> **Crawler = BFS + Queue + Visited set.**

---

16. Structured Data Extraction

- Pull specific fields (name, price, rating) from pages into JSON/DB.
- **Wrapper** = program that recognises a page's pattern and extracts target data.

Technique	Idea	Note
Manual wrapper	Hand-coded per site	Not scalable
Wrapper induction	Label few pages, system learns rules	Supervised
Automatic extraction	Cluster similar elements, derive template rules	No labels, web-scale
LLM-powered	Model reads content, outputs fields	Flexible

> **Crawl → Clean → Extract → Structure.**

---

17. Opinion Mining & Sentiment Analysis

- Extract opinions/emotions from text; decide positive / negative / neutral.
- Process: Collect → Pre-process → Feature extraction → Sentiment analysis → Summarisation.

	Opinion mining	Sentiment analysis
Scope	Broad umbrella	Subset
Output	Opinion + **about what** (aspect), strength	Only polarity

- **Types:** sentiment analysis, aspect-based (food +, service –), emotion analysis, opinion summarisation, comparative, feature-based, multimodal.
- **Techniques:** NLP (tokenize, POS, NER), pre-processing, supervised ML (SVM, Naive Bayes, DT), lexicon-based (dictionary of polarity; weak on sarcasm), deep learning (RNN, LSTM, CNN).
- **Applications:** social media, brand monitoring, customer feedback, market research, crisis management.
- **Challenges:** privacy, biased data, sensitive topics, sarcasm, negation ("not good").

> **Sentiment says "positive or negative"; opinion mining also says "about what".**

ONE-PAGE FORMULA SHEET

	Formula
Support	txns($A \wedge B$) ÷ total
Confidence	Support($A \wedge B$) ÷ Support(A)
Lift	Confidence ÷ Support(B)
PageRank	$PR(A) = (1-\beta) + \beta \times \Sigma\ PR(X)/Cout(X)$
TF	count in doc ÷ terms in doc
IDF	log10(N ÷ docs with term)
TF-IDF	TF × IDF
Cosine	$(d1 \cdot d2) \div (\|d1\| \times \|d2\|)$
SVD	$A = U\ \Sigma\ V^T$

MEMORY HOOKS

- Raw data → Patterns → Knowledge
- Support = common, Confidence = reliable, Lift = real
- Apriori BFS / Eclat DFS / FP-Growth tree, no candidates, 2 scans
- Hub = OUT, Authority = IN
- Content = ON page, Structure = BETWEEN pages, Usage = what users DO
- Tokenize → Stop words → Stem → Normalize
- Selection = choose old, Extraction = build new
- Inverted = word → docs
- LSI = DTM + SVD
- Crawler = BFS + Queue + Visited

NUMERICALS TO PRACTISE

Support/Confidence/Lift from a table · Apriori frequent itemsets · GSP join + support · PageRank 2 iterations · BoW vectors · TF-IDF for one doc · Build an inverted index.